

AI & Machine Learning in Software Testing

How five everyday testing problems get easier with ML — explained with one simple example: an online shopping app.



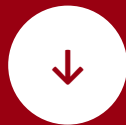
BIPEEN SINHA



Create Tests



Create Data



Prioritize



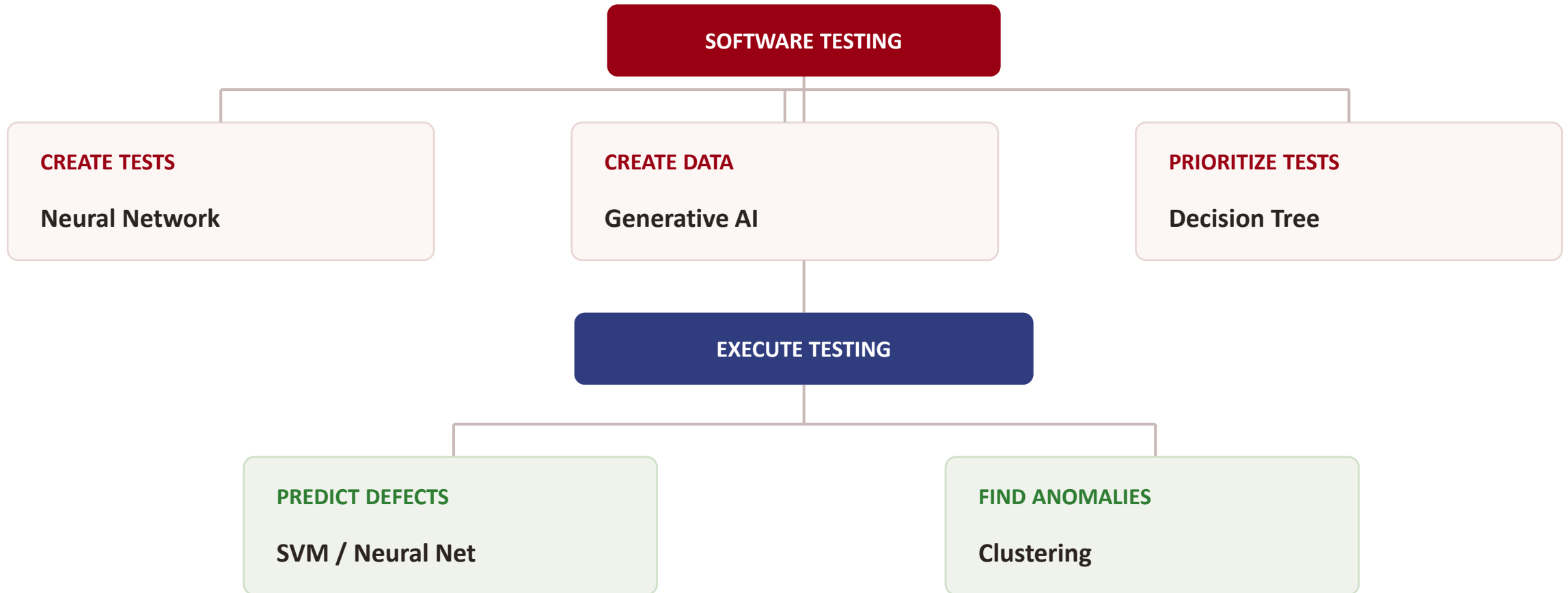
Predict Bugs



Find Anomalies

One Testing Lifecycle, Five ML Helpers

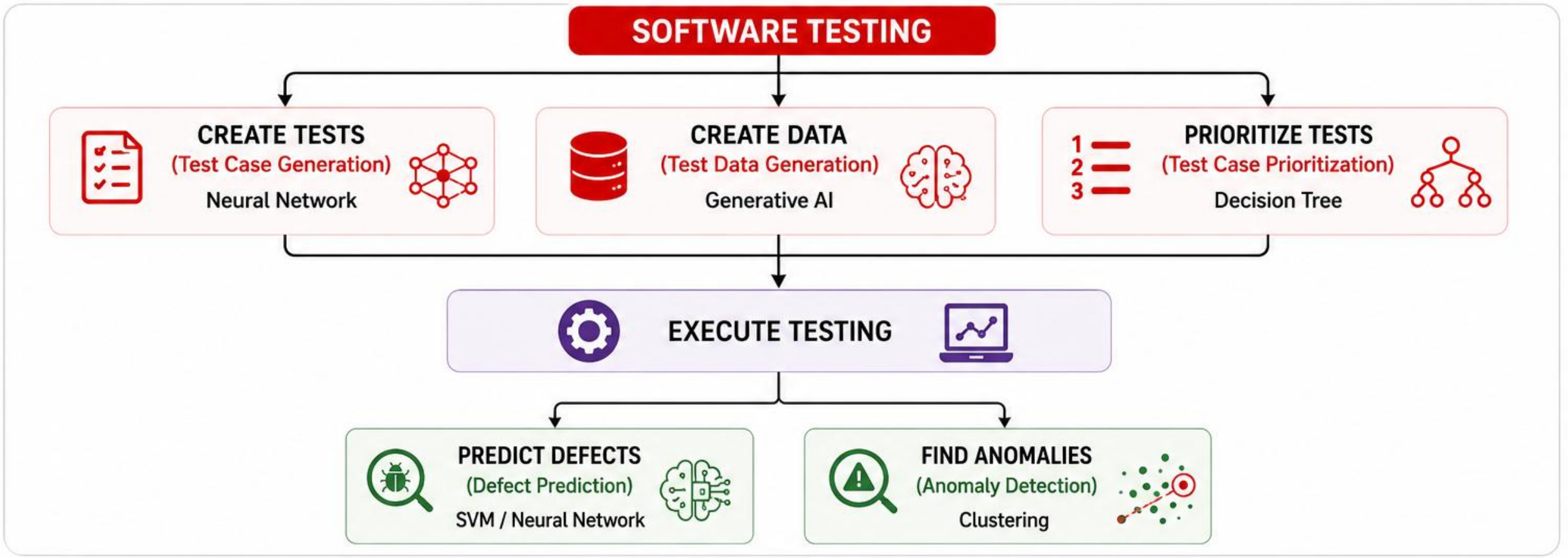
Software testing generates information. ML looks at it and helps testers decide faster.



ML doesn't replace testers — it helps them create, prioritize, predict, and detect faster.

AI / ML IN SOFTWARE TESTING – COMPLETE LIFECYCLE

From Planning to Smarter Decisions



5 WAYS ML MAKES TESTING SMARTER

	1. TEST CASE GENERATION	2. TEST DATA GENERATION	3. TEST CASE PRIORITIZATION	4. DEFECT PREDICTION	5. ANOMALY DETECTION
ML HELPS US...	 CREATE tests	 CREATE data	 DECIDE what first	 PREDICT bugs	 FIND unusual behavior
EXAMPLE	 Generate tests	 Generate test data	 Test Payment	 Payment success	 Response time

Neural Network: Creates Test Cases For You

Requirement in → Test cases out.

THE PROBLEM

Requirement: “A customer should log in using email and password.”

Input	Expected
Correct email + password	Login successful
Wrong password	Login fails
Empty email	Error

With 10,000 requirements, a human tester can't manually write every test case.

SOLUTION

Feed the model thousands of Requirement → Test Case pairs. It learns the pattern and applies it to brand-new requirements.

NEW REQUIREMENT

“Customer can reset password using registered mobile number.”



Neural Network generates the scenarios automatically:

Scenario	Expected Result
Registered mobile + valid OTP	Reset successful
Registered mobile + invalid OTP	OTP verification failed
Unregistered mobile number	Error: not registered
Empty mobile number	Error: number required



AUTOMATED TEST CASE GENERATION → NEURAL NETWORK



Neural Network learns patterns from existing test cases and generates new ones automatically.

1 WHAT IS THE PROBLEM?

Suppose our application has this requirement:

“A customer should be able to log in using email and password.”

A tester normally creates test cases like:

Test #	Input	Expected Result
1	Correct email + correct password	Login successful
2	Wrong password	Login fails
3	Wrong email	Login fails
4	Empty email	Error
5	Empty password	Error



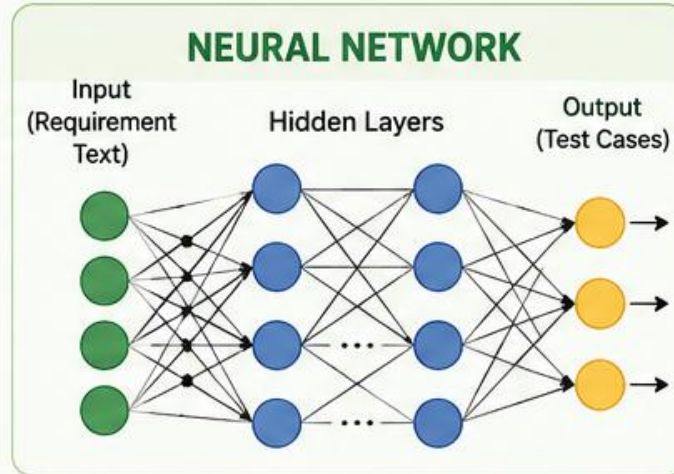
Now imagine we have **10,000** requirements. A human tester cannot manually think of every possible test case.

2 WHERE DOES ML HELP?

We give the ML model lots of examples of:

REQUIREMENT → TEST CASES

The model learns the patterns.



The neural network learns patterns from thousands of requirement → test case pairs.

3 GENERATE NEW TEST CASES

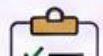
Now we give a new requirement:

“Customer can reset password using registered mobile number.”

The model understands the requirement and generates possible test cases automatically.

Test #	Input / Scenario	Expected Result
1	Registered mobile number + valid OTP	Password reset successful
2	Registered mobile number + invalid OTP	OTP verification failed
3	Unregistered mobile number	Error: Mobile number not registered
4	Valid mobile number + expired OTP	OTP expired error
5	Empty mobile number	Error: Mobile number is required
...	...	...

4 VERY SIMPLE IDEA



Requirement



Test Cases



Neural Network = learns patterns from

KEY TAKEAWAY

Generative AI: Creates Realistic Test Data

Tell it what you need — it produces the records in seconds.

NEED

1,000 customer records for a registration page.



ASK GENAI

“Generate 1,000 realistic but fake customer records.”

GOOD TEST DATA (for positive testing)

Name	Email	Age	City
Rahul	rahul@test.com	32	Mumbai
Sarah	sarah@test.com	28	London
David	david@test.com	45	Manchester

BAD / INVALID DATA (for negative testing)

rahul@ · john · @test.com · abc@

Remember: you define WHAT data you need — Generative AI creates it. Saves time, covers realistic + invalid + edge cases, improves test coverage.



TEST DATA GENERATION → GENERATIVE AI

Generative AI creates realistic test data automatically.



1 WHAT IS THE NEED?

We need to test a customer registration page.

Name

Email

Age

City

...

We need **1,000** customer records for testing.

2 MANUAL CREATION (IS DIFFICULT)

Creating 1,000 records manually takes a lot of time and effort.



Name: John
Email: john@gmail.com
Age: 35
City: London
...

3 ASK GENERATIVE AI

We ask Generative AI to create data for us.

“Generate 1,000 realistic but fake customer records for testing.”



4 AI GENERATES GOOD TEST DATA

It generates realistic and valid customer records.

Name	Email	Age	City
Rahul	rahul@test.com	32	Mumbai
Sarah	sarah@test.com	28	London
David	david@test.com	45	Manchester
...	...	...	...

✓ Useful for positive testing

5 AI GENERATES BAD / INVALID DATA

We can also ask for invalid or bad data for negative testing.

Generate invalid emails.”

Invalid Emails (Examples)

rahul@
john
@test.com
abc@
...

⚠ Useful for negative testing

VERY SIMPLE FLOW



REQUIREMENT

- (What data we need)
- Customer registration data
 - 1,000 records



GENERATIVE AI
(Creates data)



TEST DATA

- (Ready to use)
- Realistic data
 - Invalid data
 - Edge cases



GEN AI = CREATES TEST DATA

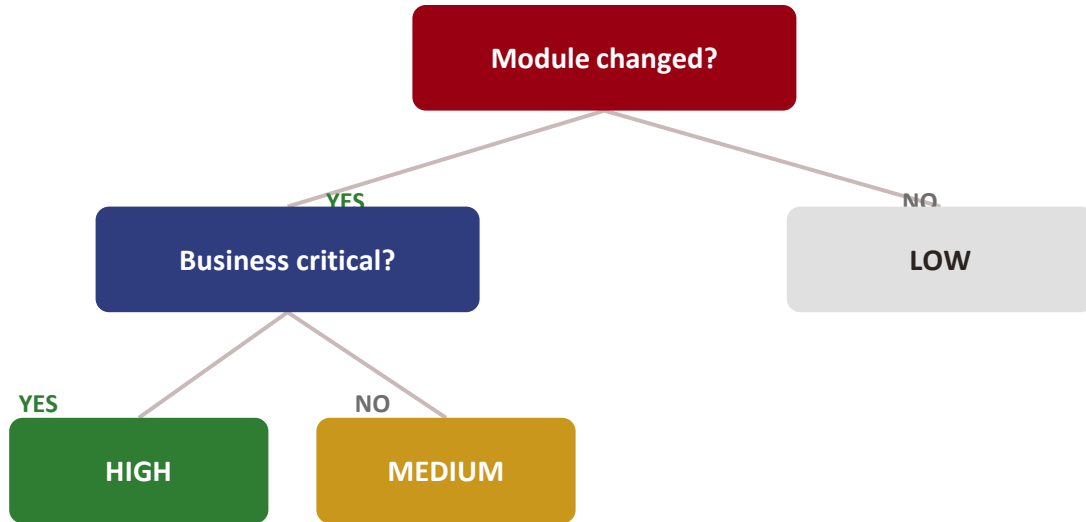
- ✓ Saves time
- ✓ More data in less time
- ✓ Realistic + invalid + edge cases
- ✓ Improves test coverage

EXAMPLE USE CASES

3 · TEST CASE PRIORITIZATION

Decision Tree: Decides What To Test First

10,000 test cases, 2 hours — which ones matter most?



Learns from history: was it changed, did it have bugs before, how often is it used, did past releases break it?

PRIORITIZED TEST CASES — RUN IN THIS ORDER

HIGH — Run First

Login · Payment

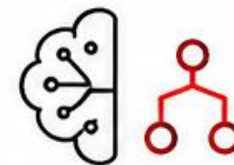
MEDIUM — Run Next

Product Search

LOW — Run Last

Change Theme (UI)

Key word: PRIORITY — focus effort on what matters most, deliver quality faster.



THE PROBLEM



10,000
TEST CASES

Not enough time to run all tests before every release.

Which test cases should we run first?

INPUTS FROM HISTORY



Was this module changed?



Did it have bugs before?



Is it business critical?

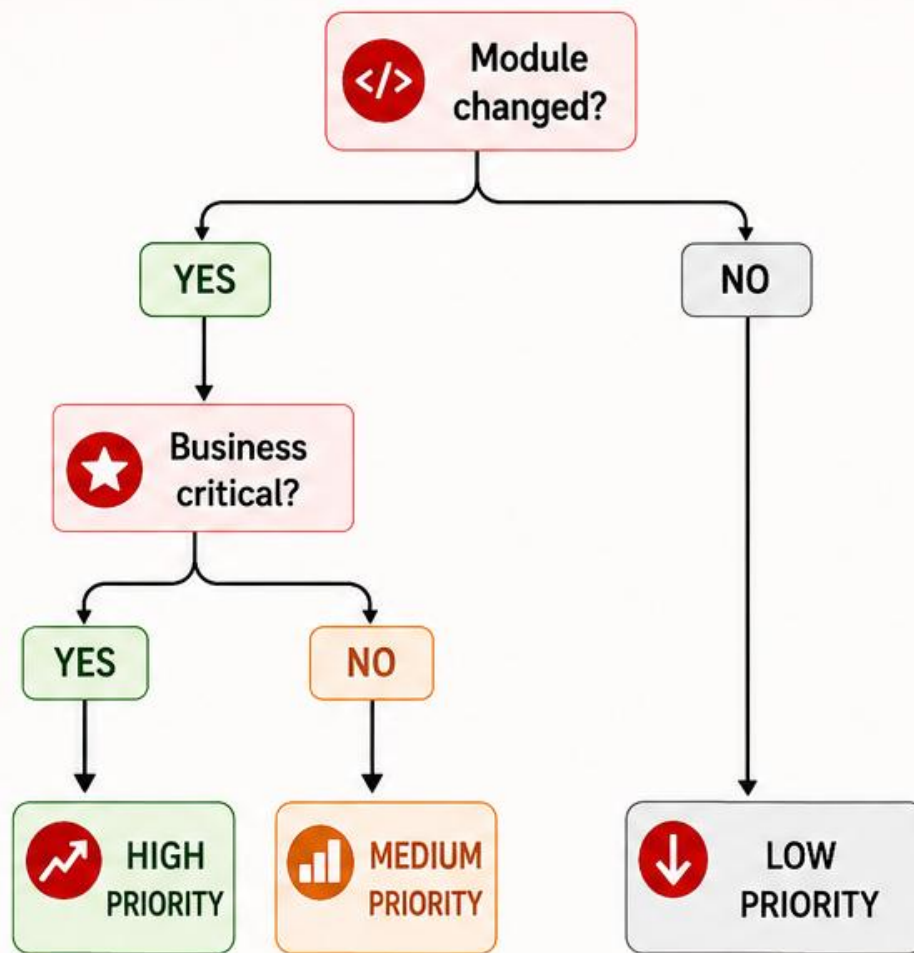


How frequently is it used?



Did previous releases break it?

DECISION TREE (ML MODEL)



The Decision Tree learns from past data and predicts which tests are most important.

PRIORITIZED TEST CASES



Run in this order

HIGH PRIORITY

Run First



Login

Changed Critical

✓ ✓



Payment

✓ ✓

MEDIUM PRIORITY

Run Next



Product Search

Changed Critical

✗ ✓

LOW PRIORITY

Run Last



Change Theme (UI)

✗ ✗



Focus effort on what matters most. Deliver quality faster.

SVM / Neural Network: Predicts Where Bugs Hide

Historical patterns reveal which modules are risky.

5 YEARS OF HISTORICAL DATA

Module	Code Changes	Past Bugs
Login	5	2
Payment	50	25
Search	8	1
Reporting	40	18

Pattern the model learns: modules with lots of code changes and lots of past bugs are more likely to contain new defects.



PAYMENT MODULE

50 code changes · 25 historical defects



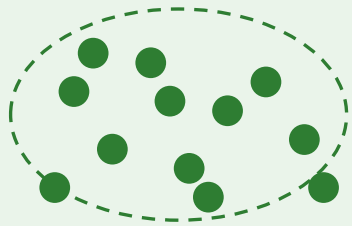
⚠ HIGH DEFECT RISK

Result: testers spend more effort here before release — catching bugs before customers do.

Clustering: Finds What's Unusual

ML learns normal behavior, then flags anything far from it.

NORMAL BEHAVIOUR

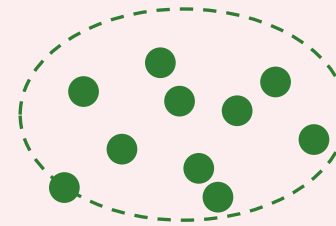


Response Time
1.2 – 1.5 sec

Transactions
~100 / min

ML groups similar behaviour together into clusters.

ANOMALY DETECTED



Response Time
12 sec (8x normal)

Transactions
5,000 / min (50x normal)

Points far from the normal clusters get flagged → alert team, investigate, fix, prevent impact.

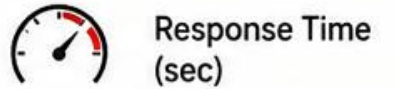
Anomaly Detection = finds something unusual or abnormal, before it becomes a customer-facing problem.

ANOMALY DETECTION → CLUSTERING

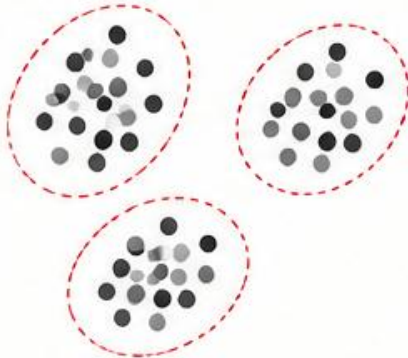
Find what's unusual. Act faster.



1 COLLECT DATA

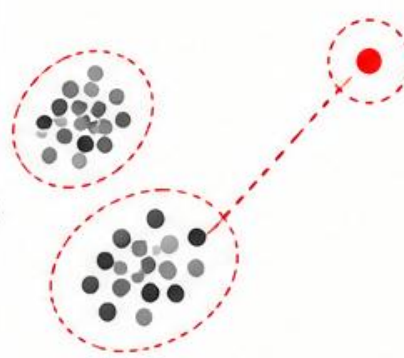


2 LEARN NORMAL (CLUSTERING)



ML groups similar behaviour together.

3 DETECT ANOMALY



Points far from clusters are flagged.

4 EXAMPLES

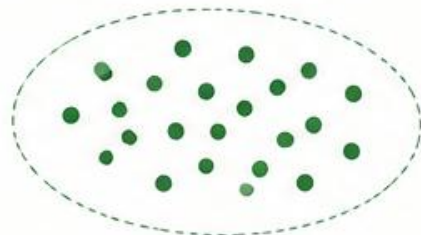


5 ALERT & ACT

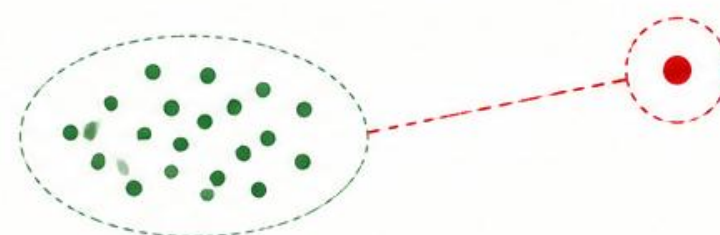


- ☒ Alert Team
- ☒ Investigate
- ☒ Fix Issue
- ☒ Prevent Impact

NORMAL BEHAVIOUR



ANOMALY DETECTED



Catch issues



Improve app



Better user



Reduce business

QUICK REFERENCE

5 Ways ML Makes Testing Smarter

One formula per problem — keep this as your cheat sheet.

Problem	Algorithm	ML Helps Us...	Example
Test Case Generation	Neural Network	CREATE tests	Generate login test cases
Test Data Generation	Generative AI	CREATE data	Generate 1,000 fake customers
Test Case Prioritization	Decision Tree	DECIDE what first	Test Payment before UI
Defect Prediction	SVM / Neural Network	PREDICT bugs	Payment module is high risk
Anomaly Detection	Clustering	FIND unusual behaviour	Response time suddenly 12 sec

In one sentence: ML helps testers CREATE, PRIORITIZE, PREDICT, and DETECT.

SUMMARY

ML Doesn't Replace Testers. It Helps Them Focus.

	Create	Neural networks turn requirements into test cases automatically.
	Generate	Generative AI produces realistic — and realistically broken — test data.
	Prioritize	Decision trees decide which tests matter most, right now.
	Predict	SVMs / neural nets flag the modules most likely to break.
	Detect	Clustering spots the behaviour that doesn't belong.

Bottom line: understand the testing problem first — the algorithm is just the tool that solves it.